GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	Algoritmos de Sincronización				
NÚMERO DE PRÁCTICA:	**02**	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026A
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		**4**
FECHA INICIO:	**20/04/2026**	**FECHA FIN:**	**24/04/2026**	**DURACIÓN:**	**2 horas**
RECURSOS A UTILIZAR: VSCode, Netbeans, Eclipse.					
DOCENTE(s): Mg. Maribel Molina Barriga					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Aplica los diferentes algoritmos relacionados con sincronización para el diseño de sistemas distribuidos.	
TEMAS: • La sincronización en Java	
COMPETENCIAS	C.a Aplica de forma transformadora conocimientos de matemática, computación eingeniería como herramienta para evaluar, sintetizar y mostrar información como fundamento de sus ideas y perspectivas para la resolución de problemas.

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

La sincronización en sistemas distribuidos es más complicada que en un sistema centralizado, ya que se debe de considerar algunos de los siguientes puntos:

- Que la información se distribuye en varias máquinas.
- Los procesos toman decisiones con base en información local.
- Se debe evitar un punto único de falla.
- No existe un reloj común, como tampoco otra fuente de tiempo global.

El tiempo es importante en un Sistema Distribuido por dos razones:

- Es una cantidad que puede medirse de manera precisa
- Existen muchos algoritmos basados en sincronización de relojes para solucionar problemas distribuidos

Pero también presenta problemas

- No existe un tiempo absoluto de referencia

- Los relojes de distintos computadores no están sincronizados

La sincronización no es trivial, porque se realiza a través de mensajes por la red, cuyo tiempo de envío puede ser variable y depender de muchos factores, como la distancia, la velocidad de transmisión o la propia saturación de la red, etc.

En la sincronización de los Sistemas Operativos Distribuidos encontramos los siguientes puntos:

1.Algoritmos para la Sincronización de Relojes

1.1. Algoritmo de Lamport
1.2. Algoritmo de Cristian
1.3. Algoritmo de Berkeley
1.4. Algoritmo con Promedio

2.Algoritmos para la Exclusión Mutua

2.1. Centralizado
2.2. Distribuido
2.3. De Anillo de Fichas (Token Ring)
2.4. De Elección
2.5. Del Grandulón (García Molina)
2.6. De Anillo

Sincronización de relojes

Para la sincronización de relojes existen las siguientes alternativas:

Relojes lógicos

- Según Lamport, la sincronización de relojes no debe ser absoluta, debido a que si dos procesos no interactúan entre sí, no requieren que sus relojes estén sincronizados.
- La distorsión de reloj es la diferencia entre los valores de tiempo de los diferentes relojes locales.
- Aquí importa el orden de ocurrencia de los eventos, no la hora exacta.

Relojes físicos

- Usan el tiempo atómico internacional (TAI) y el tiempo coordenado universal (UTC).
- Se pueden sincronizar por medio de radios de onda corta.
- También se puede usar satélite para sincronizar.

El Algoritmo de Lamport

Lamport define una relación temporal llamada: **ocurre antes de**. Por ejemplo, **(a → b)** se interpreta como **"a ocurre antes de b".** Para ilustrar esto, consideremos la siguiente situación:

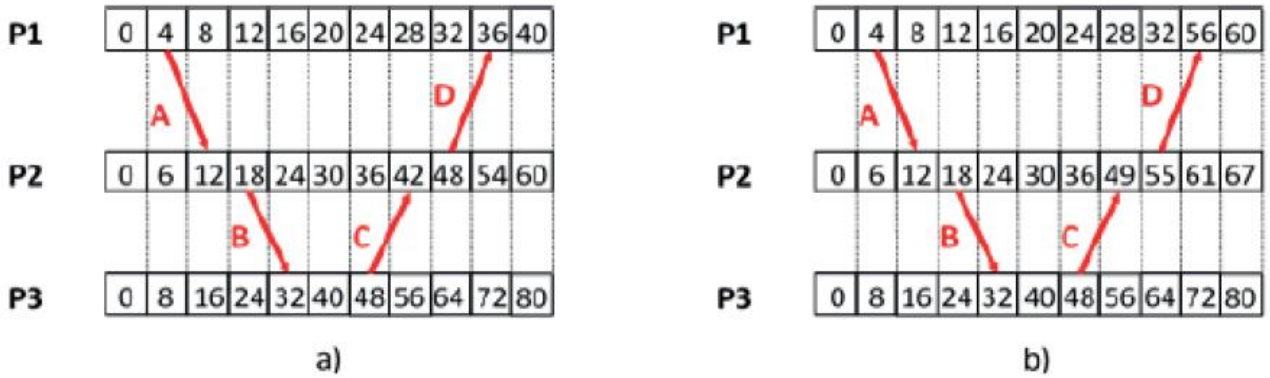
1. Si a y b son eventos del mismo proceso y a ocurre antes que b, entonces a → b es verdadero.
2. Si a es un envío de un mensaje por un proceso y b es la recepción del mensaje por otro proceso, entonces a → b es verdadero.
3. Eventos concurrentes: si a → b no es verdadero ni b → a tampoco.

Se requieren valores de tiempo para medir el tiempo:

- Si a → b, entonces C (a) < C (b).
- C siempre es creciente (se pueden hacer correcciones hacia adelante pero no hacia atrás).
- Para todos los eventos a y b, tenemos que C(a) ≠ C (b).

Un ejemplo de tres procesos (P1, P2 y P3), donde cada uno tiene su propio reloj a diferentes velocidades se muestra en la figura 5.1a, mientras que en la figura 5.1b se muestra la corrección de los relojes usando el algoritmo de Lamport.

Figura. Un ejemplo del algoritmo de Lamport en tres procesos



II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

El algoritmo de Lamport es un algoritmo de funcionamiento de relojes lógicos distribuidos. Aquí está un ejemplo de código en Java para implementar este algoritmo:

```
import java.util.ArrayList;
import java.util.List;
public class LamportClock {
     private int clock;

        public LamportClock() {
            this.clock = 0;
        }

        public synchronized int tick() {
            this.clock++;
            return this.clock;
        }

        public synchronized void update(int receivedTime) {
            this.clock = Math.max(this.clock, receivedTime) + 1;
        }

        public int getTime() {
            return this.clock;
        }

        public static void main(String[] args) {
            List<Thread> threads = new ArrayList<>();
            LamportClock clock = new LamportClock();
            for (int i = 0; i < 5; i++) {
                Thread thread = new Thread(new Runnable() {
                    @Override
                    public void run() {
                        int time = clock.tick();
                        System.out.println("Thread " + Thread.currentThread().getId() + " created
event with Lamport time " + time);
                        try {
                            Thread.sleep((long) (Math.random() * 1000));
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                        int receivedTime = clock.tick();
                        System.out.println("Thread " + Thread.currentThread().getId() + " received
event with Lamport time " + receivedTime);
                        clock.update(receivedTime);
                    }
```

```
                    });
                    threads.add(thread);
                    thread.start();
                }

                for (Thread thread : threads) {
                    try {
                        thread.join();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }

                System.out.println("Final Lamport time: " + clock.getTime());
            }
        }
```

En este código, la clase LamportClock implementa el reloj de Lamport. La función tick() aumenta el tiempo del reloj en uno y devuelve el tiempo actual. La función update() actualiza el tiempo del reloj a partir del tiempo recibido como parámetro. La función getTime()devuelve el tiempo actual del reloj.

En la función main(), se crean varios hilos, cada uno de los cuales crea un evento con el tiempo del reloj de Lamport, espera un tiempo aleatorio y luego recibe un evento con un tiempo del reloj de Lamport. El tiempo del reloj se actualiza después de recibir el evento. Al final, se imprime el tiempo del reloj de Lamport final.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Investigar, implementar, ejecutar el código de los algoritmos de manera adecuada de:
- Algoritmo de Cristian
- Algoritmo de Berkeley

Para lo cual:
- Evaluar resultados obtenidos.
- Escriba un reporte sobre las tareas realizadas y resultados.

IV. CUESTIONARIO

1. ¿Por qué es conveniente el uso de relojes lógicos en lugar de los relojes físicos?
2. Explique ¿Cuál algoritmo ya sea de Christian o de Berkeley resuelve mejor la sincronización?
3. Dado tres procesos P1, P2 y P3, representa la planificación de los procesos siguiendo el algoritmo de "ocurre antes de".

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
[3]Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
Problemas /Ejercicios propuestos / Preguntas formuladas / Resolución de casos	*Lista de cotejo*

CRITERIOS DE EVALUACIÓN

- Identifica algoritmos de sincronización global
- Utiliza las clases de sincronización en Java